AI Assistant Coding

Assignment 11.3

Name : Varsha Sri Chukka **HT. No :** 2303A52494 **Batch:** 50

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
   - o Add a contact
   - o Search for a contact
   - o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
   - o Insertion efficiency
   - o Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

Code:

```
'''
Genarte a stack class with push pop peek is_empty methods  with docsrings
''' class
Stack:
    """
    A class representing a stack data structure.

    Methods:
-     push(item): Adds an item to the top of the stack.
-     pop(): Removes and returns the item at the top of the stack. Raises an
exception if the stack is empty.
-     peek(): Returns the item at the top of the stack without removing it.
Raises an exception if the stack is empty.
-     is_empty(): Returns True if the stack is empty, False otherwise.
    """      def
__init__(self):
        """Initialize     an     empty     stack."""
self.items = []
     def push(self, item):
        """Add an item to the top of the stack."""
self.items.append(item)
```

```
    def
pop(self):
        """Remove and return the item at the top of the stack. Raises
an exception if the stack is empty."""         if self.is_empty():
            raise IndexError("Pop from an empty stack")
return self.items.pop()
    def
peek(self):
        """Return the item at the top of the stack without removing it.
Raises an exception if the stack is empty."""         if self.is_empty():
raise IndexError("Peek from an empty stack")         return self.items[-1]
    def
is_empty(self):
        """Return True if the stack is empty, False otherwise."""
return len(self.items) == 0
# Example usage: if
__name__ == "__main__":
    stack = Stack()     stack.push(1)
stack.push(2)     print(stack.peek())  #
Output: 2     print(stack.pop())   #
Output: 2     print(stack.is_empty())  #
Output: False     print(stack.pop())   #
Output: 1     print(stack.is_empty())  #
Output: True
```

Output:

```
2
2
False
1
True
```

---

ask Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

```
'''
Generate a queue class with enqueue dequeue front is_empty methods using
python lists with docstrings
''' class
Queue:
    """
    A class representing a queue data structure.

    Methods:
- enqueue(item): Adds an item to the end of the queue.
- dequeue(): Removes and returns the item at the front of the queue.
Raises an exception if the queue is empty.
- front(): Returns the item at the front of the queue without removing it.
Raises an exception if the queue is empty.
- is_empty(): Returns True if the queue is empty, False otherwise.
    """      def
__init__(self):
        """Initialize     an     empty     queue."""
self.items = []
     def enqueue(self,
item):
        """Add an item to the end of the queue."""
self.items.append(item)
     def
dequeue(self):
        """Remove and return the item at the front of the queue. Raises
an exception if the queue is empty."""         if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
return self.items.pop(0)
     def
front(self):
        """Return the item at the front of the queue without removing
it. Raises an exception if the queue is empty."""         if
self.is_empty():             raise IndexError("Front from an empty
queue")         return self.items[0]
     def
is_empty(self):
        """Return True if the queue is empty, False otherwise."""
```

```
        return len(self.items) == 0
# Example usage: if
__name__ == "__main__":
    queue = Queue()     queue.enqueue(1)
queue.enqueue(2)     print(queue.front())  #
Output: 1     print(queue.dequeue())  #
Output: 1     print(queue.is_empty())  #
Output: False     print(queue.dequeue())  #
Output: 2     print(queue.is_empty())  #
Output: True
```

Output:

```
1
1
False
2
True
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods. Sample Input Code:

class Node:

pass class

LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

Code:

```
'''
Generate a singly linked list class with insert and display methods with
docstrings
'''
```

```python
class Node:
    """
```

```
    A class representing a node in a singly linked list.

    Attributes:
- data: The value stored in the node.
- next: A reference to the next node in the list.
    """      def
__init__(self, data):
        """Initialize a node with the given data and set next to
None."""         self.data = data         self.next = None class
SinglyLinkedList:
    """
    A class representing a singly linked list.

    Methods:
-     insert(data): Inserts a new node with the given data at the end of the
list.
-     display(): Prints the elements of the list in order.
    """      def
__init__(self):
        """Initialize an empty singly linked list."""
self.head = None
     def insert(self,
data):
        """Insert a new node with the given data at the end of the
list."""         new_node = Node(data)         if self.head is None:
self.head = new_node             return         last_node = self.head
while last_node.next:
            last_node        =        last_node.next
last_node.next = new_node
     def
display(self):
        """Print the elements of the list in
order."""         current_node = self.head
while current_node:
            print(current_node.data, end=' ')
current_node = current_node.next         print()  # for a
new line after displaying the list
# Example usage: if
__name__ == "__main__":
   linked_list = SinglyLinkedList()
```

```
    linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
```

Output:

```
1 2 3
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code: class

BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.

Code:

```
'''
Generate a binary search tree with insert and inorder traversal methods
with docstrings ''' class TreeNode:
    """
    A class representing a node in a binary search tree.

    Attributes:
- data: The value stored in the node.
- left: A reference to the left child node.
- right: A reference to the right child node.
    """      def
__init__(self, data):
        """Initialize a tree node with the given data and set left and
right to None."""         self.data = data         self.left = None
self.right = None class BinarySearchTree:
    """
```

[redacted]

A class representing a binary search tree.

```
    Methods:
-     insert(data): Inserts a new node with the given data into the tree while
maintaining the binary search tree property.
-     inorder_traversal(): Returns a list of the elements in the tree in
sorted order.
    """      def
__init__(self):
        """Initialize an empty binary search tree."""
self.root = None
     def insert(self,
data):
        """Insert a new node with the given data into the tree
while maintaining the binary search tree property."""         if
self.root is None:
            self.root = TreeNode(data)
else:
            self._insert_recursive(self.root, data)
     def _insert_recursive(self, node,
data):
        """Helper method to insert a new node
recursively."""         if data < node.data:             if
node.left is None:
                node.left = TreeNode(data)
else:
                self._insert_recursive(node.left,
data)         else:             if node.right is None:
                node.right = TreeNode(data)
else:
                self._insert_recursive(node.right, data)
     def
inorder_traversal(self):
        """Return a list of the elements in the tree in sorted order."""
return self._inorder_recursive(self.root)
     def _inorder_recursive(self,
node):
        """Helper method to perform inorder traversal
recursively."""         elements = []         if node:
            elements.extend(self._inorder_recursive(node.left))
elements.append(node.data)
elements.extend(self._inorder_recursive(node.right))
return elements # Example usage: if __name__ == "__main__":
```

```
    bst = BinarySearchTree()     bst.insert(5)     bst.insert(3)
bst.insert(7)     bst.insert(2)     bst.insert(4)     bst.insert(6)
```

```
bst.insert(8)     print(bst.inorder_traversal())  # Output: [2, 3,
4, 5, 6, 7, 8]
```

Output:

```
[2, 3, 4, 5, 6, 7, 8]
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods. Sample Input Code: class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

```
def __init__(self, size=10):
    """Initialize a hash table with a specified size."""
```

```
'''
Genarate hash table with insert , search, and delete methods with docstrings
''' class
HashTable:
    """
    A class representing a hash table data structure.

    Methods:
-     insert(key, value): Inserts a key-value pair into the hash table.
- search(key): Returns the value associated with the given key. Raises an
exception if the key is not found.
-     delete(key): Removes the key-value pair associated with the given key
from the hash table. Raises an exception if the key is not found.     """
```

```
        self.size = size
self.table = [None] * self.size
     def _hash(self,
key):
        """Generate a hash for the given key."""
return hash(key) % self.size
     def insert(self, key,
value):
        """Insert a key-value pair into the hash
table."""         index = self._hash(key)         if
self.table[index] is None:
            self.table[index] = [(key, value)]         else:             for
i, (k, v) in enumerate(self.table[index]):                 if k == key:
self.table[index][i] = (key, value)  # Update existing key
return             self.table[index].append((key, value))  # Add new key-
value pair
     def search(self,
key):
        """Return the value associated with the given key. Raises an
exception if the key is not found."""         index = self._hash(key)
if self.table[index] is not None:             for k, v in self.table[index]:
if k == key:                     return v         raise KeyError("Key not
found")
     def delete(self,
key):
        """Remove the key-value pair associated with the given key from
the hash table. Raises an exception if the key is not found."""
index = self._hash(key)         if self.table[index] is not None:
for i, (k, v) in enumerate(self.table[index]):                 if k ==
key:                     del self.table[index][i]
return
        raise KeyError("Key not found")

# Example usage: if __name__ == "__main__":
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
```

```
    print(hash_table.search("age"))    # Output: 30
    hash_table.delete("name")
try:
        print(hash_table.search("name"))  # This will raise an exception
except KeyError as e:
        print(e)  # Output: Key not found
```

Output:

```
Alice
30
'Key not found'
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Code:

```
''' implement graph using adjacency
list
''' class Graph:
def __init__(self):
self.graph = {}
     def add_edge(self, u, v):
if u not in self.graph:
self.graph[u] = []          if
v not in self.graph:
self.graph[v] = []
self.graph[u].append(v)
self.graph[v].append(u)
     def display(self):
for vertex in self.graph:
            print(f"{vertex}: {self.graph[vertex]}")
# Example usage:
```

```
if __name__ == "__main__":
    g = Graph()
    g.add_edge(0, 1)
    g.add_edge(0, 2)
    g.add_edge(1, 2)
    g.add_edge(1, 3)
    g.display()
```

Output:

```
0: [1, 2]
1: [0, 2, 3]
2: [0, 1]
3: [1]
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code: class

PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Code:

```
''' implement priority queue using heapq modeule with enqueue and
dequeue operations and display methods
''' import heapq class
PriorityQueue:     def
__init__(self):
self.elements = []
     def enqueue(self, item,
priority):
        heapq.heappush(self.elements, (priority, item))
     def dequeue(self):
if not self.is_empty():




            return heapq.heappop(self.elements)[1]
raise IndexError("Priority Queue is empty")
     def
is_empty(self):
        return len(self.elements) == 0
     def
display(self):
        print("Priority Queue:")
for priority, item in self.elements:
            print(f"Item: {item}, Priority: {priority}")
# Example usage: if __name__ == "__main__":     pq =
PriorityQueue()     pq.enqueue("Task 1", 2)
pq.enqueue("Task 2", 1)     pq.enqueue("Task 3", 3)
pq.display()     print("Dequeue:", pq.dequeue())  #
Output: Task 2     pq.display()
```

Output:

```
Priority Queue:
Item: Task 2, Priority: 1
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
Dequeue: Task 2 Priority Queue:
```

```
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code: class

DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Code:

```
'''
```

```
implement a double-ended queue using collections.deque and insert and remove
from both ends with docstrings.
''' from collections import
deque class DequeDs:
    """
    A class representing a double-ended queue (deque) data structure.
    Methods:
- insert_front(item): Inserts an item at the front of the deque.
- insert_rear(item): Inserts an item at the rear of the deque.
- remove_front(): Removes and returns the item at the front of the deque.
Raises an exception if the deque is empty.
- remove_rear(): Removes and returns the item at the rear of the deque.
Raises an exception if the deque is empty.
- display(): Displays the contents of the deque.
    """      def
__init__(self):
        """Initialize     an     empty     deque."""
self.deque = deque()
     def insert_front(self,
item):
        """Insert an item at the front of the deque."""
self.deque.appendleft(item)
     def insert_rear(self,
item):
        """Insert an item at the rear of the deque."""
self.deque.append(item)
     def
remove_front(self):
        """Remove and return the item at the front of the deque. Raises
an exception if the deque is empty."""         if not self.is_empty():
            return self.deque.popleft()
raise IndexError("Deque is empty")
     def
remove_rear(self):
        """Remove and return the item at the rear of the deque. Raises
an exception if the deque is empty."""         if not self.is_empty():
            return self.deque.pop()
raise IndexError("Deque is empty")
     def
is_empty(self):
        """Check if the deque is empty."""
return len(self.deque) == 0
```

```
    def display(self):
        """Display the contents of the deque."""
print("Deque contents:", list(self.deque))
# Example usage: if __name__ == "__main__":     deque_ds = DequeDs()
deque_ds.insert_rear("Task 1")     deque_ds.insert_rear("Task 2")
deque_ds.insert_front("Task 0")     deque_ds.display()  # Output: Deque
contents: ['Task 0', 'Task 1', 'Task
2']     print("Remove front:", deque_ds.remove_front())  # Output:
Task 0     print("Remove rear:", deque_ds.remove_rear())    # Output:
Task 2     deque_ds.display()  # Output: Deque contents: ['Task 1']
print("Is deque empty?", deque_ds.is_empty())  # Output: False
deque_ds.remove_front()  # Remove remaining item     print("Is deque
empty?", deque_ds.is_empty())  # Output: True
```

Output:

```
Deque contents: ['Task 0', 'Task 1', 'Task 2']
Remove front: Task 0
Remove rear: Task 2
Deque contents: ['Task 1']
Is deque empty? False
Is deque empty? True
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Code:

```
"""Campus Resource Management application using DSA.

Modules included:
1. Student attendance tracking using a stack.
2. Event registration using a hash table.
3. Library borrowing tracker with due dates.
4. Bus scheduling and route connections using a graph.
5. Cafeteria order serving using a queue.
"""  from collections import deque
from datetime import date,
timedelta


class AttendanceTracker:
    """Track student entry and exit logs using a stack.

    Each new action (entry or exit) is pushed to the end of the
list,     which acts as a stack. The most recent activity is at the
top.     """      def __init__(self):
        """Create an empty attendance stack."""
self.attendance_log = []
     def log_entry(self,
student_name):
        """Push a student entry record onto the stack."""
self.attendance_log.append(f"{student_name} entered campus")
     def log_exit(self,
student_name):
        """Push a student exit record onto the stack."""
self.attendance_log.append(f"{student_name} exited campus")
     def
last_activity(self):
        """Return the most recent attendance activity or None."""
if not self.attendance_log:
            return None
return self.attendance_log[-1]
     def
display_daily_log(self):
        """Print the full attendance activity
log."""         print("Attendance Log:")         if
not self.attendance_log:
            print("No attendance records.")
return
```

```
        for record in
self.attendance_log:
            print(record)


class EventRegistration:
    """Manage event participants using a hash table (dictionary).

    Hash table structure:
- Key: event name
- Value: set of participant names
    """      def
__init__(self):
        """Create an empty registration table."""
self.events = {}
     def          register_participant(self,          event_name,
participant_name):
        """Add a participant to an event."""
if event_name not in self.events:
self.events[event_name] = set()
        self.events[event_name].add(participant_name)
     def          search_participant(self,          event_name,
participant_name):
        """Return True if participant is registered in the given event."""
return event_name in self.events and participant_name in
self.events[event_name]
     def remove_participant(self, event_name, participant_name):
"""Remove a participant from an event if present."""         if
self.search_participant(event_name, participant_name):

self.events[event_name].remove(participant_name)
if not self.events[event_name]:                 del
self.events[event_name]             return True
return False
     def display_event_participants(self,
event_name):         """Print all participants for
an event."""         participants =
self.events.get(event_name)         if not
participants:
            print(f"No participants found for event: {event_name}")
return         print(f"Participants in {event_name}:
{sorted(participants)}")


class LibraryManager:
    """Track available books and borrowed books with due dates."""
```

```
    def __init__(self):
        """Initialize library book collections."""
self.available_books = {}
self.borrowed_books = {}
     def add_book(self, title,
copies=1):
        """Add  one  or  more  copies  of  a  book  to  the  library."""
self.available_books[title] = self.available_books.get(title, 0) + copies
def borrow_book(self, student_name, title, borrow_days=14):
        """Borrow a book and assign a due date.

        Returns True if borrow succeeds, else False.
        """         if
self.available_books.get(title, 0) <= 0:
            return False
         self.available_books[title] -= 1         due_date =
date.today() + timedelta(days=borrow_days)
self.borrowed_books[(student_name, title)] = due_date
return True
     def return_book(self, student_name,
title):
        """Return a borrowed book and make it available
again."""         key = (student_name, title)         if key not
in self.borrowed_books:
            return False
         del self.borrowed_books[key]         self.available_books[title] =
self.available_books.get(title, 0) + 1         return True
     def
display_borrowed_books(self):
        """Print borrowed books with their due
dates."""         print("Borrowed Books:")         if
not self.borrowed_books:
            print("No borrowed books.")
return
         for (student_name, title), due_date in
self.borrowed_books.items():             print(f"{student_name} borrowed
'{title}' | Due: {due_date}")


class BusSchedulingSystem:
    """Maintain bus stop connections using a graph (adjacency list)."""
     def
__init__(self):
        """Create an empty graph of bus stops."""
```

```
        self.routes = {}
     def add_stop(self,
stop_name):
        """Add a stop to the graph if missing."""
if stop_name not in self.routes:
self.routes[stop_name] = []
     def connect_stops(self, stop_a,
stop_b):
        """Create a bidirectional connection between two
stops."""         self.add_stop(stop_a)
self.add_stop(stop_b)
         if stop_b not in
self.routes[stop_a]:
self.routes[stop_a].append(stop_b)         if
stop_a not in self.routes[stop_b]:
self.routes[stop_b].append(stop_a)
     def
display_routes(self):
        """Print all stops and directly connected
stops."""         print("Bus Route Connections:")
if not self.routes:
            print("No routes available.")
return
         for stop, connections in
self.routes.items():
            print(f"{stop} -> {connections}")


class CafeteriaOrderQueue:
    """Serve cafeteria orders in arrival order using a queue."""
     def
__init__(self):
        """Create an empty order queue."""
self.orders = deque()
     def place_order(self, student_name,
order_item):
        """Add an order to the queue."""
        self.orders.append((student_name, order_item))
     def
serve_order(self):
        """Serve and return the next order in FIFO order."""
if not self.orders:
            return None
return self.orders.popleft()
     def
display_pending_orders(self):
        """Print all pending orders in queue
order."""         print("Pending Cafeteria Orders:")
if not self.orders:
```

```
            print("No pending orders.")
return
         for student_name, order_item in
self.orders:             print(f"{student_name}:
{order_item}")


def demo_attendance_tracker():
    """Example usage for AttendanceTracker."""
print("\n1) Attendance Tracker Demo")     tracker =
AttendanceTracker()     tracker.log_entry("Alice")
tracker.log_entry("Bob")
tracker.log_exit("Alice")
tracker.display_daily_log()     print("Last
activity:", tracker.last_activity())


def demo_event_registration():
    """Example usage for EventRegistration."""
print("\n2) Event Registration Demo")     event_system =
EventRegistration()
event_system.register_participant("Hackathon", "Alice")
event_system.register_participant("Hackathon", "Bob")
event_system.register_participant("Music Fest", "Charlie")
     print("Is Bob in Hackathon?",
event_system.search_participant("Hackathon",
"Bob"))
event_system.remove_participant("Hackathon", "Bob")
event_system.display_event_participants("Hackathon")


def demo_library_manager():
    """Example usage for LibraryManager."""
print("\n3) Library Manager Demo")
library = LibraryManager()
library.add_book("Data Structures", 2)
library.add_book("Python Basics", 1)

    success = library.borrow_book("Alice", "Data Structures", borrow_days=7)
print("Borrow Data Structures for Alice:", success)
library.display_borrowed_books()

    library.return_book("Alice", "Data Structures")
library.display_borrowed_books()


def demo_bus_scheduling_system():
    """Example usage for BusSchedulingSystem."""
```

```
    print("\n4) Bus Scheduling Demo")     bus_system =
BusSchedulingSystem()
bus_system.connect_stops("Main Gate", "Library")
bus_system.connect_stops("Library", "Hostel")
bus_system.connect_stops("Main Gate", "Cafeteria")
bus_system.display_routes()


def demo_cafeteria_order_queue():
    """Example usage for CafeteriaOrderQueue."""
print("\n5) Cafeteria Queue Demo")     queue_system
= CafeteriaOrderQueue()
queue_system.place_order("Alice", "Sandwich")
queue_system.place_order("Bob", "Coffee")
queue_system.place_order("Charlie", "Pasta")
     queue_system.display_pending_orders()
print("Served:", queue_system.serve_order())
queue_system.display_pending_orders()


if __name__ == "__main__":
demo_attendance_tracker()
demo_event_registration()     demo_library_manager()
demo_bus_scheduling_system()
demo_cafeteria_order_queue()
```

Output:

```
1) Attendance Tracker Demo
Attendance Log:
Alice entered campus
Bob entered campus
Alice exited campus
Last activity: Alice exited campus
2) Event Registration Demo
Is Bob in Hackathon? True
Participants in Hackathon: ['Alice']
3) Library Manager Demo
Borrow Data Structures for Alice: True
Borrowed Books:
Alice borrowed 'Data Structures' | Due: 2026-03-17
Borrowed Books:
No borrowed books.
```

```
4)    Bus Scheduling Demo
Bus Route Connections:
Main Gate -> ['Library', 'Cafeteria']
Library -> ['Main Gate', 'Hostel']
Hostel -> ['Library']
Cafeteria -> ['Main Gate']

5)    Cafeteria Queue Demo
Pending Cafeteria Orders:
Alice: Sandwich
Bob: Coffee
Charlie: Pasta
Served: ('Alice', 'Sandwich')
Pending Cafeteria Orders:
Bob: Coffee
Charlie: Pasta
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

```
"""Smart Hospital Management System using DSA.

Features:
1. Patient Check-In System using Queue (FIFO).
2. Emergency Case Handling using Priority Queue.
3. Medical Records Storage using Hash Table.
4. Doctor Appointment Scheduling sorted by time.
5. Hospital Room Navigation using Graph.
"""
```

```
from collections import deque
import heapq


class PatientCheckInSystem:
    """Register and treat patients in order of arrival using a queue."""
     def
__init__(self):
        """Initialize an empty patient queue."""
self.patient_queue = deque()
     def check_in(self,
patient_name):
        """Add a patient to the check-in queue."""
self.patient_queue.append(patient_name)
     def
treat_next_patient(self):
        """Treat and return the next patient in FIFO order."""
if not self.patient_queue:
            return None         return
self.patient_queue.popleft()
     def display_waiting_patients(self):
"""Display all waiting patients."""
print("Waiting Patients:")         if not
self.patient_queue:
            print("No patients waiting.")
return         for patient in
self.patient_queue:
            print(patient)


class EmergencyCaseHandler:
    """Handle emergency patients using a priority queue.

    Lower priority value means higher treatment priority.
    Example:
-     1: Critical     - 2: High
-     3: Moderate
    """      def
__init__(self):
        """Initialize an empty emergency priority
queue."""         self.emergency_heap = []
self.counter = 0
     def add_emergency_case(self, patient_name, priority):
"""Add an emergency patient with a priority level."""
self.counter += 1
```

```
        heapq.heappush(self.emergency_heap, (priority, self.counter,
patient_name))
    def
treat_emergency_case(self):
        """Treat and return the highest-priority emergency patient."""
if not self.emergency_heap:
            return None
        _, _, patient_name = heapq.heappop(self.emergency_heap)
return patient_name
    def
display_emergency_queue(self):
        """Display emergency queue ordered by treatment
priority."""         print("Emergency Queue:")         if not
self.emergency_heap:
            print("No emergency cases.")
return
         for priority, _, patient_name in sorted(self.emergency_heap):
            print(f"Priority {priority}: {patient_name}")


class MedicalRecordsStorage:
    """Store and retrieve medical records quickly using a hash table."""
    def
__init__(self):
        """Initialize   empty   records   table   by   patient   ID."""
self.records = {}
    def add_record(self, patient_id,
patient_details):
        """Add or update a patient record."""
self.records[patient_id] = patient_details
    def get_record(self,
patient_id):
        """Retrieve patient record by ID, or None if not found."""
return self.records.get(patient_id)
    def remove_record(self,
patient_id):
        """Delete a patient record by ID. Returns True if
removed."""         if patient_id in self.records:             del
self.records[patient_id]             return True         return
False


class DoctorAppointmentScheduler:
    """Schedule and manage doctor appointments sorted by time."""
    def
__init__(self):
        """Initialize empty appointment list."""
```

```
        self.appointments = []
     def     add_appointment(self,     time_slot,     patient_name,
doctor_name):
        """Add an appointment and keep the list time-sorted.
         time_slot format: HH:MM (24-
hour)
        """         self.appointments.append((time_slot, patient_name,
doctor_name))         self.appointments.sort(key=lambda item: item[0])
     def
next_appointment(self):
        """Return the next appointment without removing
it."""         if not self.appointments:             return
None         return self.appointments[0]
     def
display_appointments(self):
        """Display all appointments sorted by
time."""         print("Doctor Appointments:")
if not self.appointments:
            print("No appointments scheduled.")
return
         for time_slot, patient_name, doctor_name in self.appointments:
print(f"{time_slot} - {patient_name} with Dr. {doctor_name}")


class HospitalRoomNavigation:
    """Represent ward and room connections using a graph."""
     def
__init__(self):
        """Initialize   an   empty   graph   as   adjacency   list."""
self.graph = {}
     def add_room(self,
room_name):
        """Add a room/ward node if it does not exist."""
if room_name not in self.graph:
self.graph[room_name] = []
     def connect_rooms(self, room_a,
room_b):
        """Create two-way connection between
rooms/wards."""         self.add_room(room_a)
self.add_room(room_b)
         if room_b not in
self.graph[room_a]:
self.graph[room_a].append(room_b)         if
room_a not in self.graph[room_b]:
self.graph[room_b].append(room_a)
```

```
    def get_connected_rooms(self, room_name):
        """Return all directly connected rooms for the given room."""
return self.graph.get(room_name, [])
     def display_map(self):
        """Display the full hospital room connection map."""
print("Hospital Room Navigation Map:")         if not
self.graph:
            print("No room connections available.")
return
         for room, connected in self.graph.items():
            print(f"{room} -> {connected}")


def demo_patient_check_in():
    """Example usage of PatientCheckInSystem."""
print("\n1) Patient Check-In Demo")     check_in =
PatientCheckInSystem()     check_in.check_in("Alice")
check_in.check_in("Bob")     check_in.check_in("Charlie")
check_in.display_waiting_patients()
print("Treating:", check_in.treat_next_patient())
check_in.display_waiting_patients()


def demo_emergency_cases():
    """Example usage of EmergencyCaseHandler."""
print("\n2) Emergency Case Handling Demo")     emergency =
EmergencyCaseHandler()
emergency.add_emergency_case("Patient-X", 2)
emergency.add_emergency_case("Patient-Y", 1)
emergency.add_emergency_case("Patient-Z", 3)
emergency.display_emergency_queue()
    print("Treating emergency:", emergency.treat_emergency_case())
emergency.display_emergency_queue()


def demo_medical_records():
    """Example usage of MedicalRecordsStorage."""     print("\n3) Medical
Records Storage Demo")     records = MedicalRecordsStorage()
records.add_record("P101", {"name": "Alice", "age": 21, "diagnosis":
"Flu"})          records.add_record("P102",  {"name":  "Bob",  "age":  30,
"diagnosis":
"Allergy"})     print("Record P101:",
records.get_record("P101"))
records.remove_record("P102")
```

```
    print("Record P102:", records.get_record("P102"))


def demo_appointments():
    """Example usage of DoctorAppointmentScheduler."""
print("\n4) Doctor Appointment Scheduling Demo")
scheduler = DoctorAppointmentScheduler()
scheduler.add_appointment("11:30", "Charlie", "Mehta")
scheduler.add_appointment("09:15", "Alice", "Verma")
scheduler.add_appointment("10:00", "Bob", "Khan")
scheduler.display_appointments()     print("Next
appointment:", scheduler.next_appointment())


def demo_room_navigation():
    """Example usage of HospitalRoomNavigation."""     print("\n5)
Hospital Room Navigation Demo")     navigation = HospitalRoomNavigation()
navigation.connect_rooms("Reception", "Ward-A")
navigation.connect_rooms("Ward-A", "Ward-B")
navigation.connect_rooms("Ward-B", "ICU")
navigation.connect_rooms("Ward-A", "Lab")     navigation.display_map()
print("Connected to Ward-A:", navigation.get_connected_rooms("Ward-A"))


if __name__ == "__main__":
demo_patient_check_in()
demo_emergency_cases()
demo_medical_records()
demo_appointments()
demo_room_navigation()
```

Output:

```
1)    Patient Check-In Demo Waiting Patients:
Alice
Bob
Charlie
Treating: Alice Waiting
Patients:
Bob
Charlie

2)    Emergency Case Handling Demo

Emergency Queue:
Priority 1: Patient-Y
```

```
Priority 2: Patient-X
Priority 3: Patient-Z
Treating emergency: Patient-Y
Emergency Queue:
Priority 2: Patient-X
Priority 3: Patient-Z

3) Medical Records Storage Demo
Record P101: {'name': 'Alice', 'age': 21, 'diagnosis': 'Flu'}
Record P102: None

4) Doctor Appointment Scheduling Demo
Doctor Appointments:
09:15 - Alice with Dr. Verma
10:00 - Bob with Dr. Khan
11:30 - Charlie with Dr. Mehta
Next appointment: ('09:15', 'Alice', 'Verma')

5) Hospital Room Navigation Demo
Hospital Room Navigation Map:
Reception -> ['Ward-A']
Ward-A -> ['Reception', 'Ward-B', 'Lab']
Ward-B -> ['Ward-A', 'ICU']
ICU -> ['Ward-B']
Lab -> ['Ward-A']
Connected to Ward-A: ['Reception', 'Ward-B', 'Lab']
```

Task Description #11: Smart City Traffic Control System A city

plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Code:

```
"""Smart City Traffic Control System using DSA.
```

```
Features:
1. Traffic Signal Queue using Queue (FIFO).
2. Emergency Vehicle Handling using Priority Queue.
3. Vehicle Registration Lookup using Hash Table.
4. Road Network Mapping using Graph.
5. Parking Slot Availability using Stack.
"""  from collections import
deque import heapq


class TrafficSignalQueue:
    """Manage vehicles waiting at a traffic signal using a queue (FIFO)."""
     def
__init__(self):
        """Initialize    an    empty    vehicle    queue."""
self.queue = deque()
     def add_vehicle(self,
vehicle_no):
        """Add a vehicle to the end of the queue."""
self.queue.append(vehicle_no)
     def
pass_vehicle(self):
        """Allow the first vehicle in queue to pass the
signal."""         if not self.queue:             return None
return self.queue.popleft()
     def
display_queue(self):
        """Display vehicles currently waiting at the
signal."""         print("Traffic Signal Queue:")         if
not self.queue:
            print("No vehicles
waiting.")             return
for vehicle in self.queue:
            print(vehicle)


class EmergencyVehiclePriorityHandling:
    """Prioritize emergency vehicles using a priority queue.

    Lower priority value means higher urgency:
- 1: Ambulance
- 2: Fire truck
- 3: Police or other emergency services
    """
```

```
    def __init__(self):
        """Initialize an empty emergency
heap."""         self.heap = []
self.counter = 0
     def     add_emergency_vehicle(self,     vehicle_no,     vehicle_type,
priority):
        """Add an emergency vehicle to the priority queue."""
self.counter += 1         heapq.heappush(self.heap, (priority,
self.counter, vehicle_no, vehicle_type))
     def
dispatch_vehicle(self):
        """Dispatch and return the highest-priority emergency
vehicle."""         if not self.heap:             return None
        _, _, vehicle_no, vehicle_type = heapq.heappop(self.heap)
return vehicle_no, vehicle_type
     def
display_priority_queue(self):
        """Display emergency vehicles in treatment/dispatch
order."""         print("Emergency Vehicle Priority Queue:")
if not self.heap:
            print("No emergency vehicles waiting.")
return
         for priority, _, vehicle_no, vehicle_type in
sorted(self.heap):             print(f"Priority {priority}:
{vehicle_type} ({vehicle_no})")


class VehicleRegistrationLookup:
    """Store and retrieve vehicle details quickly using hash table."""
     def
__init__(self):
        """Initialize registration records."""
self.records = {}
     def      register_vehicle(self,      vehicle_no,      owner_name,
vehicle_type):
        """Add or update vehicle registration
details."""         self.records[vehicle_no] = {
"owner": owner_name,
            "type": vehicle_type,
        }
     def get_vehicle_details(self,
vehicle_no):
        """Fetch vehicle details by registration number."""
return self.records.get(vehicle_no)
     def remove_vehicle(self,
vehicle_no):
        """Remove a vehicle record. Returns True if removed."""
```

```
        if vehicle_no in self.records:
del self.records[vehicle_no]
return True         return False


class RoadNetworkMapping:
    """Represent roads and intersections as a graph."""
     def
__init__(self):
        """Initialize    empty    road    network    graph."""
self.graph = {}
     def add_intersection(self,
node):
        """Add an intersection if not already
present."""         if node not in self.graph:
self.graph[node] = []
     def connect_intersections(self, a,
b):
        """Create a two-way road between
intersections."""         self.add_intersection(a)
self.add_intersection(b)
         if b not in
self.graph[a]:
self.graph[a].append(b)         if
a not in self.graph[b]:
self.graph[b].append(a)
     def
show_network(self):
        """Display all intersections and their connected
roads."""         print("Road Network Map:")         if not
self.graph:
            print("No road connections available.")
return
         for intersection, neighbors in
self.graph.items():
            print(f"{intersection} -> {neighbors}")


class ParkingSlotAvailability:
    """Track parking slots using stack operations.

    The top of stack contains the most recently freed slot.
    """      def __init__(self,
total_slots):
        """Initialize stack with available parking slot IDs."""
self.available_slots = list(range(total_slots, 0, -1))
self.occupied = {}
```

```
    def park_vehicle(self,
vehicle_no):
        """Assign top available slot to vehicle. Returns slot number
or None."""         if not self.available_slots:
            return None         slot =
self.available_slots.pop()
self.occupied[vehicle_no] = slot
return slot
    def unpark_vehicle(self,
vehicle_no):
        """Free slot of vehicle and push it back to available
stack."""         if vehicle_no not in self.occupied:
return None         slot = self.occupied.pop(vehicle_no)
self.available_slots.append(slot)         return slot
    def
display_parking_status(self):
        """Display current occupied and available parking
slots."""         print("Parking Status:")
print("Occupied:", self.occupied)         print("Available slots
stack:", self.available_slots)


def demo_traffic_signal_queue():
    """Example usage of TrafficSignalQueue."""
print("\n1) Traffic Signal Queue Demo")
signal = TrafficSignalQueue()
signal.add_vehicle("TN01AB1234")
signal.add_vehicle("TN02XY4567")
signal.add_vehicle("TN03CD8901")
signal.display_queue()
    print("Passed vehicle:", signal.pass_vehicle())
signal.display_queue()


def demo_emergency_handling():
    """Example usage of EmergencyVehiclePriorityHandling."""
print("\n2) Emergency Vehicle Priority Handling Demo")
emergency = EmergencyVehiclePriorityHandling()
emergency.add_emergency_vehicle("EM001", "Fire Truck", 2)
emergency.add_emergency_vehicle("EM002", "Ambulance", 1)
emergency.add_emergency_vehicle("EM003", "Police", 3)
emergency.display_priority_queue()     print("Dispatched:",
emergency.dispatch_vehicle())
emergency.display_priority_queue()
```

```
def demo_registration_lookup():
    """Example usage of VehicleRegistrationLookup."""
print("\n3) Vehicle Registration Lookup Demo")     lookup =
VehicleRegistrationLookup()
lookup.register_vehicle("TN10AA1001", "Arun", "Car")
lookup.register_vehicle("TN10BB2002", "Nisha", "Bike")
print("TN10AA1001:", lookup.get_vehicle_details("TN10AA1001"))
lookup.remove_vehicle("TN10BB2002")     print("TN10BB2002:",
lookup.get_vehicle_details("TN10BB2002"))


def demo_road_network():
    """Example usage of RoadNetworkMapping."""
print("\n4) Road Network Mapping Demo")     network =
RoadNetworkMapping()
network.connect_intersections("Junction-A", "Junction-B")
network.connect_intersections("Junction-B", "Junction-C")
network.connect_intersections("Junction-A", "Junction-D")
network.show_network()


def demo_parking_slots():
    """Example usage of ParkingSlotAvailability."""     print("\n5)
Parking Slot Availability Demo")     parking =
ParkingSlotAvailability(total_slots=3)     print("Park TN22ZZ1111 at
slot:", parking.park_vehicle("TN22ZZ1111"))     print("Park TN22ZZ2222
at slot:", parking.park_vehicle("TN22ZZ2222"))
parking.display_parking_status()     print("Unpark TN22ZZ1111 from
slot:", parking.unpark_vehicle("TN22ZZ1111"))
parking.display_parking_status()


if __name__ == "__main__":
    demo_traffic_signal_queue()
demo_emergency_handling()
demo_registration_lookup()
demo_road_network()     demo_parking_slots()
```

Output:

```
1) Traffic Signal Queue Demo
Traffic Signal Queue:
TN01AB1234
TN02XY4567
TN03CD8901
Passed vehicle: TN01AB1234
```

```
Traffic Signal Queue:
TN02XY4567
TN03CD8901

2) Emergency Vehicle Priority Handling Demo
Emergency Vehicle Priority Queue:
Priority 1: Ambulance (EM002)
Priority 2: Fire Truck (EM001)
Priority 3: Police (EM003)
Dispatched: ('EM002', 'Ambulance')
Emergency Vehicle Priority Queue:
Priority 2: Fire Truck (EM001)
Priority 3: Police (EM003)

3) Vehicle Registration Lookup Demo
TN10AA1001: {'owner': 'Arun', 'type': 'Car'}
TN10BB2002: None

4) Road Network Mapping Demo
Road Network Map:
Junction-A -> ['Junction-B', 'Junction-D']
Junction-B -> ['Junction-A', 'Junction-C']
Junction-C -> ['Junction-B']
Junction-D -> ['Junction-A']

5) Parking Slot Availability Demo
Park TN22ZZ1111 at slot: 1
Park TN22ZZ2222 at slot: 2
Parking Status:
Occupied: {'TN22ZZ1111': 1, 'TN22ZZ2222': 2}
Available slots stack: [3] Unpark
TN22ZZ1111 from slot: 1
Parking Status:
Occupied: {'TN22ZZ2222': 2}
Available slots stack: [3, 1]
```

Task 12: An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Code:

```
"""Smart E-Commerce Platform using core data structures.

Modules:
1. Shopping Cart Management using Stack (with undo).
2. Order Processing using Queue (FIFO).
3. Top-Selling Products Tracker using Priority Queue.
4. Product Search Engine using Hash Table.
5. Delivery Route Planning using Graph.
"""  from collections import
deque import heapq


class ShoppingCartManager:
    """Manage cart operations using a stack and support undo actions."""
     def
__init__(self):
        """Initialize cart stack and operation history
stack."""         self.cart_stack = []         self.history =
[]
     def add_product(self,
product_name):
        """Add product to cart and save operation for undo."""
self.cart_stack.append(product_name)         self.history.append(("add",
product_name))
     def
remove_product(self):
        """Remove the latest product from cart and save operation for
undo."""         if not self.cart_stack:
            return None         product =
self.cart_stack.pop()
self.history.append(("remove", product))
return product
     def
undo_last_action(self):
        """Undo the most recent add/remove operation."""
if not self.history:
            return "Nothing to undo"

        action, product = self.history.pop()         if action ==
"add":             if self.cart_stack and self.cart_stack[-1] ==
product:
                self.cart_stack.pop()
return f"Undo add: {product}"

self.cart_stack.append(product)
```

```
        return f"Undo remove: {product}"
     def
display_cart(self):
        """Display current cart
products."""         print("Shopping
Cart:")         if not self.cart_stack:
            print("Cart is empty.")
return         for product in
self.cart_stack:
            print(product)


class OrderProcessingSystem:
    """Process orders in first-come-first-served order using queue."""
     def
__init__(self):
        """Initialize an empty order queue."""
self.orders = deque()
     def place_order(self, order_id,
customer_name):
        """Place a new order in queue."""
self.orders.append((order_id, customer_name))
     def
process_order(self):
        """Process next order from queue."""
if not self.orders:
            return None
return self.orders.popleft()
     def display_pending_orders(self):
"""Display all pending orders."""
print("Pending Orders:")         if not
self.orders:
            print("No pending orders.")
return         for order_id, customer_name in
self.orders:             print(f"{order_id} -
{customer_name}")


class TopSellingProductsTracker:
    """Track product popularity using priority queue based on sales count."""
     def
__init__(self):
        """Initialize sales map and max-heap container."""
self.sales_count = {}
     def record_sale(self, product_name,
count=1):         """Increase sales count of a
product."""
```

```
        self.sales_count[product_name] = self.sales_count.get(product_name, 0)
+ count      def get_top_products(self, top_n=3):
        """Return top N products by sales using a priority
queue."""         max_heap = [(-count, product) for product, count
in self.sales_count.items()]         heapq.heapify(max_heap)
         result = []         for _ in
range(min(top_n, len(max_heap))):
count, product = heapq.heappop(max_heap)
result.append((product, -count))         return
result


class ProductSearchEngine:
    """Provide fast product lookup by product ID using hash table."""
     def
__init__(self):
        """Initialize product table."""
self.products = {}
     def add_product(self, product_id, name,
price):         """Add or update product details
by ID."""         self.products[product_id] = {
            "name": name,
            "price": price,
        }
     def search_product(self,
product_id):
        """Return product details by ID, or None."""
return self.products.get(product_id)
     def remove_product(self,
product_id):
        """Remove a product by ID. Returns True if
removed."""         if product_id in self.products:
del self.products[product_id]             return True
return False


class DeliveryRoutePlanner:
    """Model delivery network of warehouses and locations as graph."""
     def __init__(self):         """Initialize
empty adjacency list graph."""         self.network
= {}
```

```
    def add_location(self, location):
"""Add location node if missing."""
if location not in self.network:
self.network[location] = []
     def connect_locations(self, source,
destination):
        """Create bi-directional route between two locations."""
self.add_location(source)         self.add_location(destination)
         if destination not in
self.network[source]:
self.network[source].append(destination)         if
source not in self.network[destination]:
self.network[destination].append(source)
     def get_connected_locations(self,
location):
        """Return all direct connections from a location."""
return self.network.get(location, [])
     def
display_network(self):
        """Display complete delivery network
graph."""         print("Delivery Route Network:")
if not self.network:
            print("No routes available.")
return         for location, links in
self.network.items():
            print(f"{location} -> {links}")


def demo_shopping_cart():
    """Example usage for ShoppingCartManager."""
print("\n1) Shopping Cart Management Demo")
cart = ShoppingCartManager()
cart.add_product("Laptop")
cart.add_product("Headphones")
cart.add_product("Mouse")     cart.display_cart()
    print("Removed:", cart.remove_product())
print(cart.undo_last_action())     cart.display_cart()


def demo_order_processing():
    """Example usage for
OrderProcessingSystem."""     print("\n2) Order
Processing Demo")     orders =
OrderProcessingSystem()
orders.place_order("O101", "Alice")
orders.place_order("O102", "Bob")
orders.place_order("O103", "Charlie")
```

```
    orders.display_pending_orders()
print("Processed order:", orders.process_order())
orders.display_pending_orders()


def demo_top_selling_tracker():
    """Example usage for TopSellingProductsTracker."""
print("\n3) Top-Selling Products Tracker Demo")     tracker =
TopSellingProductsTracker()     tracker.record_sale("Laptop",
15)     tracker.record_sale("Headphones", 25)
tracker.record_sale("Mouse", 10)
tracker.record_sale("Keyboard", 20)     print("Top 3
products:", tracker.get_top_products(top_n=3))


def demo_product_search_engine():
    """Example usage for ProductSearchEngine."""     print("\n4)
Product Search Engine Demo")     search_engine =
ProductSearchEngine()     search_engine.add_product("P1001",
"Laptop", 75000)     search_engine.add_product("P1002",
"Smartphone", 35000)     print("Search P1001:",
search_engine.search_product("P1001"))
search_engine.remove_product("P1002")     print("Search P1002:",
search_engine.search_product("P1002"))


def demo_delivery_route_planner():
    """Example usage for DeliveryRoutePlanner."""
print("\n5) Delivery Route Planning Demo")     planner =
DeliveryRoutePlanner()
planner.connect_locations("Warehouse-A", "City-Center")
planner.connect_locations("Warehouse-A", "Zone-1")
planner.connect_locations("Zone-1", "Zone-2")
planner.connect_locations("City-Center", "Zone-3")
planner.display_network()     print("Connected to
Warehouse-A:",
planner.get_connected_locations("Warehouse-A"))


if __name__ == "__main__":
demo_shopping_cart()
demo_order_processing()
demo_top_selling_tracker()
demo_product_search_engine()
demo_delivery_route_planner()
```

Output:

```
1) Shopping Cart Management Demo
Shopping Cart:
Laptop
Headphones
Mouse
Removed: Mouse
Undo remove: Mouse
Shopping Cart:
Laptop
Headphones
Mouse

2) Order Processing Demo
Pending Orders:
O101 - Alice
O102 - Bob
O103 - Charlie
Processed order: ('O101', 'Alice')
Pending Orders:
O102 - Bob
O103 - Charlie

3) Top-Selling Products Tracker Demo
Top 3 products: [('Headphones', 25), ('Keyboard', 20), ('Laptop', 15)]

4) Product Search Engine Demo
Search P1001: {'name': 'Laptop', 'price': 75000}
Search P1002: None

5) Delivery Route Planning Demo
Delivery Route Network:
Warehouse-A -> ['City-Center', 'Zone-1']
City-Center -> ['Warehouse-A', 'Zone-3']
Zone-1 -> ['Warehouse-A', 'Zone-2']
Zone-2 -> ['Zone-1']
Zone-3 -> ['City-Center']
Connected to Warehouse-A: ['City-Center', 'Zone-1']
```